
ADVANCED IMAGE CONVERTER WITH WEBP SUPPORT

B Pardhu*¹

*¹Student, CSE (DS), Ballari Institute Of Technology & Management, Ballari, Karnataka, India.

DOI: <https://www.doi.org/10.56726/IRJMETS98646>

ABSTRACT

The proliferation of diverse image formats in contemporary computing environments has increased the demand for efficient and dependable image conversion solutions. Although traditional formats such as PNG and JPEG are widely supported, newer formats like WebP provide improved compression efficiency but lack consistent support in offline desktop tools. This paper describes the development of a Java-based desktop image conversion system capable of processing multiple image formats, including PNG, JPEG, BMP, and WebP. The application is implemented using Java Swing for the graphical interface and the ImageIO framework enhanced through plugin integration. A modular layered architecture is employed to separate presentation, processing, storage, and auxiliary operations, thereby improving extensibility and maintainability. Special consideration is given to transparency handling during format conversion to ensure visual correctness. Experimental observations indicate that the system performs stable and accurate offline image conversion and remains functional when distributed as a standalone executable.

Keywords: Image Processing, Java Imageio, Webp Format, Desktop Application, Modular Architecture.

I. INTRODUCTION

Images represent a core data type in modern software systems, spanning applications such as digital media management, web interfaces, graphical visualization, and computer vision. The continuous growth in image resolution and usage volume has intensified the need for storage-efficient and high-quality image formats. Among recent developments, the WebP format has emerged as a strong alternative to conventional formats due to its ability to reduce file size while maintaining acceptable visual fidelity.

Despite these advantages, WebP adoption in desktop environments remains limited, particularly for offline tools. Many available image converters rely on online platforms, which introduces privacy risks, network dependency, and inconsistent performance. Consequently, there exists a practical need for an offline, lightweight image conversion solution that supports both legacy and modern image formats.

Java provides a portable execution environment and includes the ImageIO API for image manipulation. However, default ImageIO support is restricted to a small set of formats. By leveraging ImageIO's plugin-based extensibility, additional formats can be supported without sacrificing platform independence. This work presents a Java desktop application that extends ImageIO functionality to enable reliable multi-format image conversion, including WebP support.

II. METHODOLOGY

The application utilizes the ImageIO service provider interface to dynamically discover and register available image codecs at runtime. A dedicated initialization routine ensures that all required plugins, including those supporting WebP, are correctly registered when the application starts. This mechanism is essential for maintaining format compatibility when the application is packaged as a standalone executable.

During conversion from formats supporting transparency to those that do not, the system performs a controlled color model transformation. The original image is rendered onto an RGB background to eliminate alpha channel conflicts, thereby preserving visual clarity and preventing conversion failures. To ensure responsiveness and thread safety, the graphical interface is initialized through the Swing event dispatch mechanism.

III. SYSTEM ARCHITECTURE

The proposed system adopts a layered architectural model to ensure clear separation of responsibilities and to facilitate maintenance and future enhancements.

A. Presentation Layer

The presentation layer is implemented using Java Swing and manages all user interactions. It allows users to select image files, preview visual content, choose target formats, and initiate conversion operations. The interface is constructed using standard graphical components to maintain simplicity and cross-platform consistency.

B. Processing Layer

The processing layer encapsulates the image conversion logic. It coordinates reading input images, performing format transformations, and applying necessary preprocessing steps. This layer also addresses format compatibility issues, particularly those related to transparency when exporting images to formats that do not support alpha channels.

C. Storage Layer

The storage layer handles interaction with the file system. It uses ImageIO readers and writers to load images from disk and persist converted images. This abstraction decouples file input-output operations from the processing logic.

D. Support Layer

The support layer consists of utility components responsible for file validation, format verification, and output file extension management. These functions improve system robustness by preventing invalid operations and enforcing consistency.

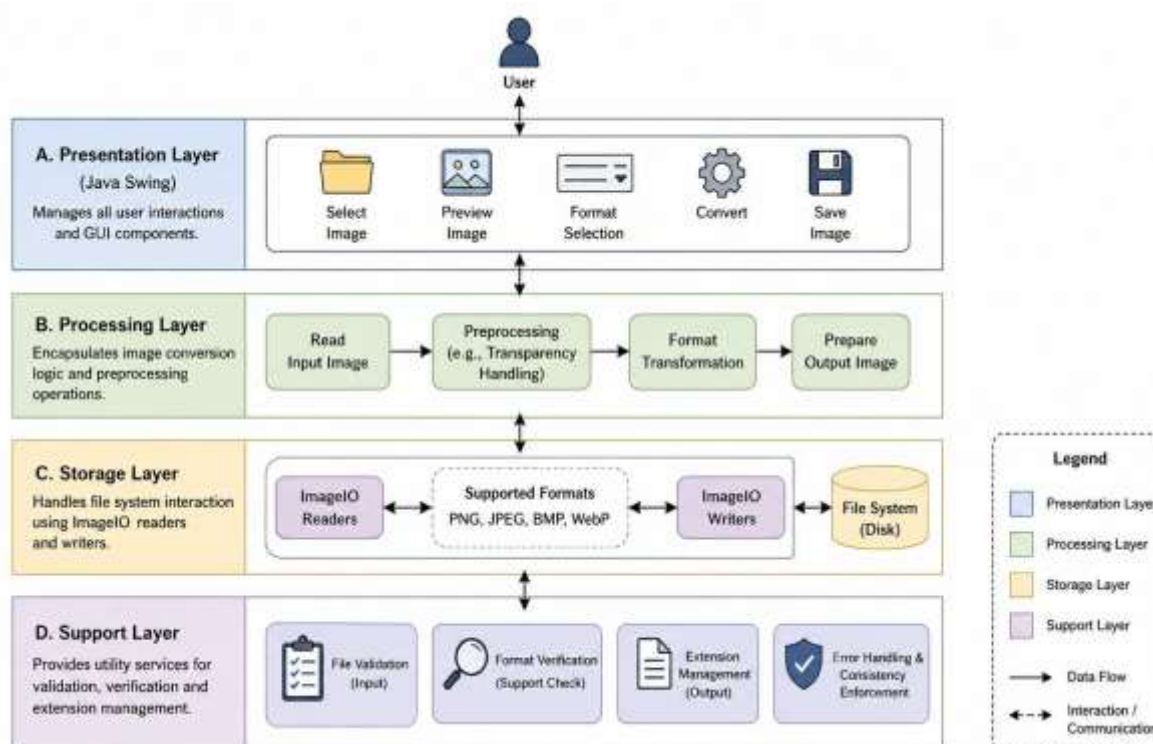


Figure 1: Layered Architecture of Advanced Image Converter System

IV. RESULTS AND DISCUSSION

The system was evaluated using a range of image files differing in resolution, format, and color composition. The experimental results demonstrate successful conversion between all supported formats, including WebP. Image previews were displayed accurately, and the converted outputs retained consistent visual quality.

Additional testing confirmed that the application functions reliably in offline conditions and remains fully operational when packaged as a native executable. The absence of external runtime dependencies contributes to predictable behavior and ease of deployment.



Fig 2: Home screen

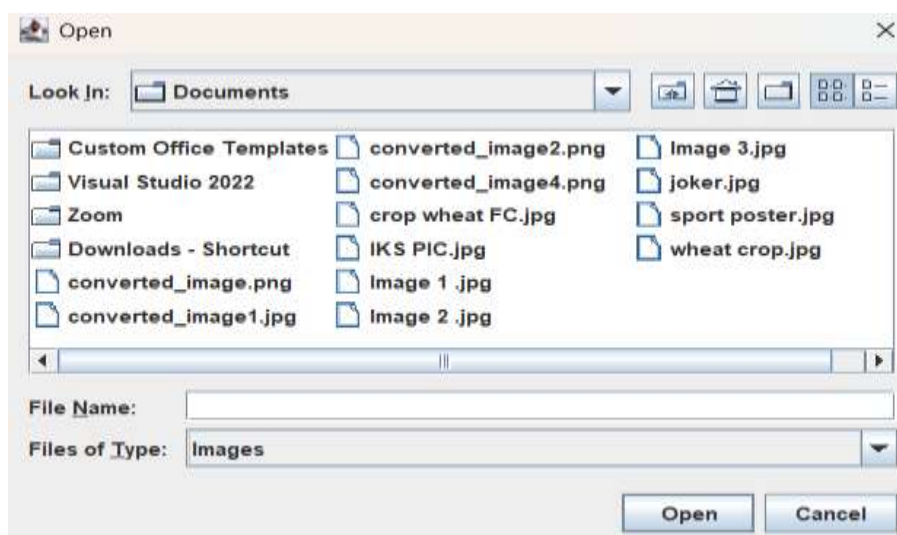


Fig 3: Image selection window



Fig 4: Image preview



Fig 5: Converted image output

V. CONCLUSION

This paper presented a Java-based desktop image conversion framework that extends the ImageIO API to support modern image formats such as WebP. The modular layered design enhances maintainability and scalability, while offline operability ensures data privacy and consistent performance. Experimental results validate the effectiveness of the proposed approach for practical desktop image conversion tasks.

Future work may explore batch processing capabilities, user-defined compression parameters, and integration of additional image codecs.

ACKNOWLEDGEMENTS

The author sincerely thanks Ms. Anushya V. P, Department of Computer Science and Engineering (Data Science), for her guidance and technical support throughout this project. Appreciation is also extended to Dr. Aradhana D, Head of the Department and Project Coordinator, for her encouragement and support. The author is grateful to Dr. Yadavalli Basavaraj, Principal of the institution, for providing the necessary facilities. Thanks are also due to the faculty members of the department for their assistance.

VI. REFERENCES

- [1] J. Gosling, B. Joy, G. Steele, and G. Bracha, The Java Language Specification. Oracle, 2014.
- [2] Oracle, "Java Image I/O API Guide," Oracle Documentation, 2024.
- [3] Google Developers, "WebP Image Format Specification," 2023.
- [4] S. Grange et al., "Efficient image compression using WebP," Google Research, 2015.